

WEBRUM-01: Web RUM

Fundamentals

Series: WEBRUM — Web Real User Monitoring | **Notebook:** 1 of 10 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

Real User Monitoring (RUM) captures the actual experience of users interacting with your web applications. Unlike synthetic monitoring, which simulates user interactions from controlled locations, RUM measures real browser performance, user behavior, and errors as they happen. Dynatrace RUM uses a lightweight JavaScript agent (OneAgent RUM injection) to automatically capture page loads, user actions, XHR/fetch calls, JavaScript errors, and Core Web Vitals — all flowing into Grail for DQL analysis.

Sprint 1.337 (April 2026): `frontend.name` Now Permission-Relevant

Sprint 1.337 added `frontend.name` as a **permission-relevant field** on RUM metrics, sessions, and Smartscape entities in Grail. This means IAM policies can now scope record-level and field-level access by frontend application, eliminating the need to split RUM data into separate buckets purely for access control.

Practical impact for WEBRUM authors:

- When you instrument a new application, choose `frontend.name` deliberately — it now drives **both observability** (queries, dashboards, segments) **and access control** (which teams can see the data).
 - Stable, hyphenated identifiers work best: `checkout-web`, `account-web`, `public-marketing`. Avoid spaces, version numbers in the name, or environment suffixes (those belong on `dt.security_context` or environment fields).
 - Existing tenants automatically gain the field — no re-instrumentation required. Update IAM policies (see **IAM-04 § Sprint 1.337**) to take advantage.
-

Table of Contents

1. [How Dynatrace RUM Works](#)
 2. [RUM vs Synthetic Monitoring](#)
 3. [RUM Data Model](#)
 4. [Exploring User Sessions](#)
 5. [Exploring User Actions](#)
 6. [Session and Action Counts](#)
 7. [Summary and Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	At least one web application with RUM injection active
Permissions	<code>storage:events:read</code> , <code>storage:entities:read</code>
Data	At least 24 hours of RUM session data

1. How Dynatrace RUM Works

Dynatrace captures real user experience through a **JavaScript agent** that is automatically injected into web pages by OneAgent. The injection process works as follows:

1. **OneAgent instruments the web server** — When a browser requests a page, OneAgent on the server-side injects a small JavaScript snippet into the HTML response.
2. **The JavaScript agent loads** — The injected snippet bootstraps the full RUM JavaScript agent from the Dynatrace CDN or your ActiveGate.

3. **Browser events are captured** — The agent hooks into browser APIs (Performance API, Navigation Timing, Resource Timing, PerformanceObserver) to capture timing data, user interactions, and errors.
4. **Data is sent to Dynatrace** — Captured data is batched and sent as beacons to the Dynatrace cluster via the beacon endpoint.
5. **Data lands in Grail** — Sessions, actions, and errors are stored as structured records queryable via DQL.

Injection Methods

Method	How It Works	Best For
Automatic injection	OneAgent modifies HTML responses at the web server	Traditional server-rendered apps
Manual injection	Developer adds the RUM script tag manually	CDN-hosted content, SPAs with no server-side agent
Agentless RUM	JavaScript tag added without full OneAgent	Environments where OneAgent cannot be deployed

Note: Automatic injection is the recommended approach as it requires zero code changes and keeps the JavaScript agent version in sync with OneAgent.

2. RUM vs Synthetic Monitoring

RUM and synthetic monitoring are complementary approaches — each answers different questions:

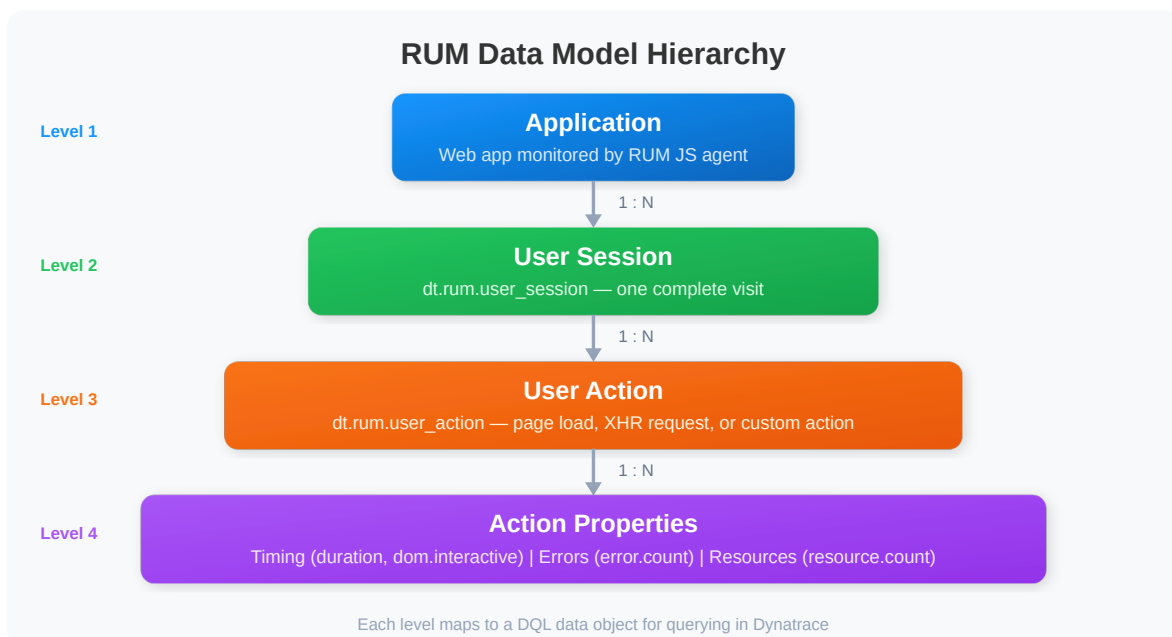
Aspect	RUM	Synthetic
Data source	Real user browsers	Simulated browsers from controlled locations
Coverage	Only when real users visit	Runs on schedule, even at 3 AM
Variability	High — reflects real network conditions, devices, geographies	Low — consistent execution environment

Aspect	RUM	Synthetic
Baseline	Organic — based on actual usage patterns	Fixed — ideal for SLA monitoring
Error detection	Catches errors users actually encounter	Catches errors in scripted flows
Performance data	Real-world load times with real content	Clean-room load times (no cache, extensions)
Geographic insight	Wherever real users are located	Only from configured locations

Tip: Use synthetic for availability SLAs and baseline benchmarks. Use RUM for understanding the actual user experience and identifying issues affecting real users.

3. RUM Data Model

Dynatrace RUM data is organized into a hierarchical model:



Key Data Objects

DQL Data Object	Description	Key Fields
<code>user.sessions</code>	A complete user visit from first page to last	<code>sessionId</code> , <code>userType</code> , <code>duration</code> , <code>userActionCount</code> , <code>totalErrorCount</code>
<code>user.events</code>	A single user interaction (page load, click, XHR)	<code>action.name</code> , <code>action.type</code> , <code>duration</code> , <code>dom.interactive.time</code>
<code>user.events</code>	JavaScript or XHR error	<code>error.message</code> , <code>error.type</code> , <code>error.source</code>
<code>smartscapeNodes</code> <code>"FRONTEND"</code>	The monitored application entity — one node per configured web or mobile app	<code>name</code> , <code>frontend.name</code> , <code>frontend.type</code> , <code>id_classic</code>

The Application Entity: `FRONTEND`

Everything else in the table above is *session and action data*. The last row is the **entity** — the configured application those sessions belong to. On the classic surface that was `dt.entity.application` ; on Smartscape it is the `FRONTEND` node type (model `dt.smartscape.frontend` , titled *Digital Experience Frontend*), which holds web **and** mobile applications, discriminated by `frontend.type` .

```
smartscapeNodes "FRONTEND"  
| filter frontend.type == "web"  
| fields name, frontend.name, id, id_classic, tags  
| sort name asc
```

SaaS 1.344 (released 07/27/2026, with a **staged tenant rollout from 07/29/2026**) is what makes this the primary entity surface: the Digital Experience apps — Experience Vitals, Users & Sessions, Error Inspector, Synthetic — moved so that their queries and filters "primarily use `dt.smartscape.*` entities, instead of `dt.entity.*` and `dt.rum.application.*` ". Verify 1.344 has reached your tenant before depending on the app-side change. The node type is queryable via DQL either way, and classic `fetch dt.entity.application` still works — keep it as a genuine fallback during the rollout.

Three details worth knowing before you use the node:

- **name VS frontend.name**. `name` is the display name (`Beta - Elevate`); `frontend.name` is the normalized form determined at ingest (`Beta_-_Elevate`). Use `name` for reporting, `frontend.name` when matching against ingest-side configuration.
- **id_classic is the join key**. It holds the `APPLICATION-*` `id` (or `MOBILE_APPLICATION-*` for a mobile app), so dashboards and saved queries still written against `dt.entity.application` can be reconciled with migrated ones.
- **frontend.type filters correctly but is absent from the model's own fields array**. The Semantic Dictionary's field list is an index, not a completeness guarantee — do not conclude the field is missing because it is not enumerated.

This is orthogonal to the vocabulary question in WEBRUM-09 — read them as answering different questions. WEBRUM-09 asks which **session-field** vocabulary your tenant serves: classic RUM on Grail (`userActionCount` , `totalErrorCount` , `browserFamily`) or New RUM (`user_action_count` , `error.count` , `browser.name`). That answer governs every session and action query in the rest of this series. `FRONTEND` is a change to the **entity** vocabulary and is independent of it — it is how you enumerate and identify applications whichever session model your tenant serves. Answer WEBRUM-09's question for your session queries; use `FRONTEND` for your entity queries. Neither decision constrains the other.

Frontend-to-Backend Topology (SaaS 1.344)

Two 1.344 changes connect this RUM data to the backend side of the platform:

- **The Smartscape service dependency graph now displays FRONTEND nodes**, letting you "drill down the call path from user-facing applications to the services they depend on, all in one view" — frontend and service topology were previously separate pictures.
- **Distributed tracing can explore frontend events and sessions without switching apps**.

The Semantic Dictionary exposes the linkage itself as `frontend.link` — *"a frontend-backend tracing link that connects a span to a frontend user event and/or user session"*, explicitly distinct from a span link, which references other spans. Two cautions before building on it: the field is `frontend.link` (dotted), and its declared stability is **experimental** — not yet a foundation for a production dashboard or alert.

Until 1.344 reaches your tenant, the working correlation path is unchanged — and it keeps working afterwards: the `x-dtc` header the RUM agent propagates on XHR/fetch

requests links a user action to the resulting server-side trace (WEBRUM-02 covers that instrumentation), supplemented by session-level joins on the application and session identifiers.

The topology side belongs to the tracing series. SPANS-04 is the better home for the service-dependency-graph and trace-exploration mechanics; this note exists so a reader arriving from RUM knows the linkage exists and what it is called.

Sources: [Dynatrace SaaS release notes 1.344 \(DT docs\)](#). **Derived:** the `frontend.link` name, type and `experimental.stability` are read from `dt.semantic_dictionary.fields` on a live tenant (07/30/2026); the "until 1.344 lands" fallback combines that with the `x-dtc` propagation documented in WEBRUM-02.

Session Types

User Type	Description
REAL_USER	Identified or anonymous real visitor
ROBOT	Bot or crawler detected by Dynatrace
SYNTHETIC	Synthetic monitor execution

Note: Always filter for `userType == "REAL_USER"` when analyzing genuine user experience to exclude bots and synthetic executions.

4. Exploring User Sessions

Let's start by examining what a user session looks like in Grail. A session represents a single visit — from the first page load to when the user closes the tab or the session times out (30 minutes of inactivity by default).

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//  userType -&gt; dt.rum.user_type      userAgentCount -&gt;
user_action_count
//  totalErrorCount -&gt; error.count      sessionId -&gt; dt.rum.session.id
//  hasSessionReplay -&gt; characteristics.has_replay
//  browserFamily -&gt; browser.name      osFamily -&gt; os.name
//  application -&gt; primary_tags.application
//  country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//  screen.width|height -&gt; browser.window.width|height
//  dom.interactive.time -&gt; performance.dom_interactive
//  load.event.time -&gt; performance.load_event_end
//  server.time -&gt; ttfb.waiting_duration
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//  * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing –
//  the "real_user" literal itself could NOT be confirmed here and is the
documented value form;
//  * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Explore recent user sessions – sample 10 sessions to see available fields
fetch user.sessions, from:-1h
| filter dt.rum.user_type == "real_user"
| fieldsKeep dt.rum.session.id, primary_tags.application, dt.rum.user_type,
duration, user_action_count, error.count, geo.city.name, geo.country.name,
os.name, browser.name
| sort duration desc
| limit 10
```

The query above returns the 10 longest sessions from the last hour. Each record includes the session ID, application name, duration, action count (number of interactions), error count, and geographic/device information.

Session Duration Distribution

Understanding how long sessions last helps identify engagement patterns:


```
// Session duration distribution – bucket sessions by duration range
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| fieldsAdd duration_sec = duration / 1s
| fieldsAdd duration_bucket = if(duration_sec < 10, "< 10s",
  else: if(duration_sec < 30, "10-30s",
  else: if(duration_sec < 60, "30-60s",
  else: if(duration_sec < 300, "1-5 min",
  else: "> 5 min"))))
| summarize session_count = count(), by:{duration_bucket}
| sort session_count desc
```

5. Exploring User Actions

User actions represent individual interactions within a session. Dynatrace automatically detects three types of actions:

Action Type	Description	Example
Load	Full page load or navigation	User navigates to <code>/checkout</code>
XHR	Background XHR/fetch request triggered by interaction	Click triggers API call
Custom	Developer-defined action via RUM API	Custom business event

Let's explore the most common user actions:

```
// Field vocabulary corrected 08/12/2026 – this series targets **New RUM**,
// but was written
// against names that are null on New RUM data, so these cells returned
// nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
// javascript agent):
//   action.type == "Load"           -&gt; characteristics.classifier ==
//   "page_summary"
//   action.type                     -&gt; user_action.type
//   (hard_navigation | same_view)
//   action.name                     -&gt; page.detected_name
//   web_vitals.largest_contentful_paint -&gt; lcp.start_time      (327,099
//   populated)
//   web_vitals.cumulative_layout_shift -&gt; cls.value           (387,254
//   populated)
//   app.name                        -&gt; dt.rum.application.id
// UNITS CHANGE WITH THE FIELD. web_vitals.* was a nanosecond DURATION, so `/
// 1ms` was correct for
// it; lcp.start_time is a PLAIN NUMBER already in milliseconds, and dividing
// it by 1ms yields
// null. Compare it against the 2500/4000 ms thresholds directly.
// `web_vitals.*` does still exist in the same schema, but carried 16 records
// in 30 days against
// lcp.*'s 327,099 – it is not a different RUM generation, just a rarely-
// populated sibling.
// Top 20 user actions by count in the last hour
fetch user.events, from:-1h
| summarize action_count = count(), avg_duration = avg(duration), by:
{page.detected_name, user_action.type}
| sort action_count desc
| limit 20
```

Page Load Actions with Timing Breakdown

For load-type actions, Dynatrace captures detailed timing milestones:

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
// filter type == "Error" -&gt; filter characteristics.has_error == true
(11,909 events; identical
// population to isNotNull(error.type), whose
values are request/csp/exception)
// error.message -&gt; error.reason
// user_action.type == "RouteChange" -&gt; "same_view" (the New RUM SPA
route-change value; the
// only other value is "hard_navigation".
"Custom" has NO equivalent.)
// connection.type -&gt; network.protocol.name
// CLASSIFIER MATTERS AS MUCH AS THE FIELD: navigation-timing fields
(performance.dom_interactive,
// performance.load_event_end) live on classifier "navigation" and are 0 on
"page_summary", so a
// page_summary filter silently empties them. ttfb.* is the opposite – it
lives on page_summary.
// Page load timing breakdown – average timings for the top 10 pages
fetch user.events, from:-1h
| filter characteristics.classifier == "navigation"
| summarize action_count = count(),
    avg_duration = avg(duration),
    avg_dom_interactive = avg(performance.dom_interactive),
    avg_load_event = avg(performance.load_event_end),
    by:{page.detected_name}
| sort action_count desc
| limit 10
```

6. Session and Action Counts

Tracking session and action volume over time reveals traffic patterns, peak hours, and trend changes.

```
// Session count over the last 24 hours, bucketed by hour
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| makeTimeseries session_count = count(), interval:1h
```

```
// Action volume by type over the last 24 hours
fetch user.events, from:-24h
| makeTimeseries action_count = count(), interval:1h, by:{user_action.type}
```

```
// Sessions and actions per application – high-level overview
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize session_count = count(),
    total_actions = sum(user_action_count),
    total_errors = sum(error.count),
    avg_duration = avg(duration),
    by:{primary_tags.application}
| sort session_count desc
```

7. Summary and Next Steps

In this notebook, we covered:

- **How RUM works** — JavaScript agent injection, browser event capture, beacon delivery to Grail
- **RUM vs synthetic** — Complementary approaches for full UX visibility
- **RUM data model** — Sessions, actions, and errors as hierarchical records in Grail
- **Basic DQL queries** — Exploring sessions, actions, timing breakdowns, and volume trends

Next Steps

- **WEBRUM-02: SPA Instrumentation** — Deep dive into Single-Page Application monitoring with Angular, React, and Vue.js
- **WEBRUM-03: Core Web Vitals** — Track LCP, INP, and CLS with DQL

References

- [Dynatrace RUM Overview](#)
- [RUM JavaScript version \(DT docs\)](#)
- [Monitor web performance with DQL \(DT docs\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.